

Advanced Computer Networks Lab



Dr Sudipta Saha

Associate Professor

Dept of Computer Science & Engineering

Indian Institute of Technology Bhubaneswar

Lab Assignment-3

Teaching Assistants: Satwik and Arindam



**DSSRG: Decentralized
Smart Systems Research
Group**

<https://sites.google.com/iitbbs.ac.in/dssrg>

Or Google dssrg iitbbs



Introduction

- What will be there in assignment 3
 - Video streaming
 - Multiplayer video game server
 - File transfer
- What is the goal of assignment 3
 - Where UDP are used?
 - Applications which use UDP
 - How UDP is used in applications
- Goal of assignment 3.1 and 3.2
 - How UDP is used for video streaming?
 - For reading video and extracting frames, we will use OpenCV library
 - How to transfer and parse data in binary instead of plain text



Table of contents

- A3.1 and 3.2
 - Transport Layer Protocols: UDP and TCP
 - Why do we even need UDP?
 - UDP socket programming
 - C++ and CMake
 - Video Streaming
 - Using OpenCV
 - Encoding frames to reduce size
 - Sending and parsing binary data



Transport Layer Protocols

- TCP and UDP
- TCP is connection oriented, ensures data integrity through ordering, acknowledgement and retransmission.
- UDP is fast, connectionless, and unreliable protocol.



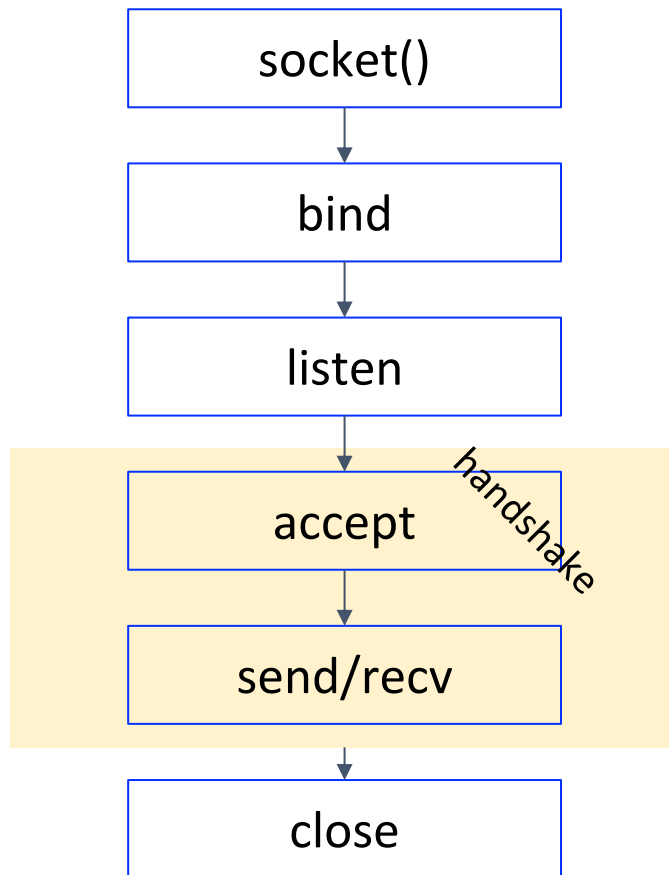
Why do we even need UDP?

- TCP has everything, why UDP then?
- UDP gives speed and a blank slate.
- We can implement additional functionality as required.

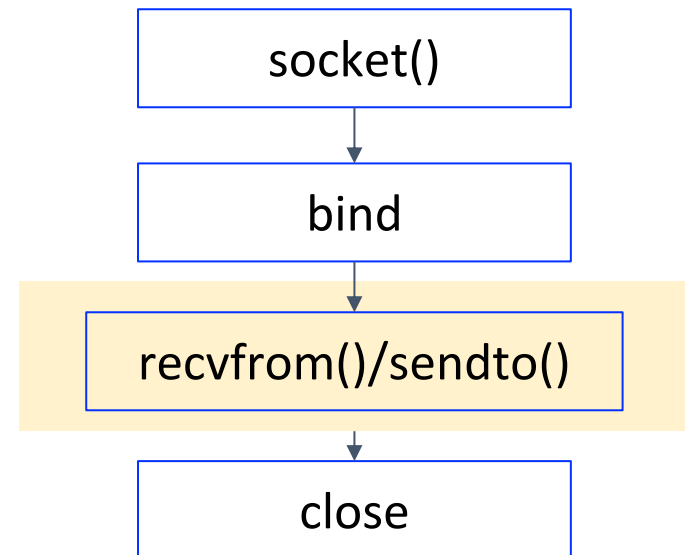


UDP socket programming (server)

TCP

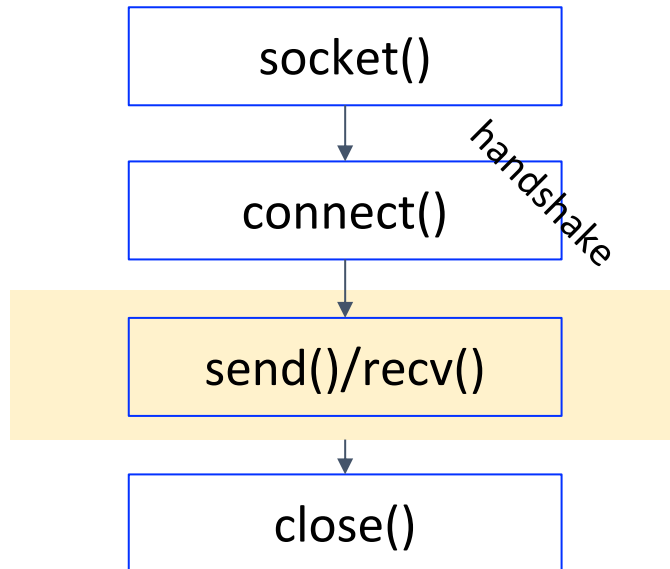


UDP

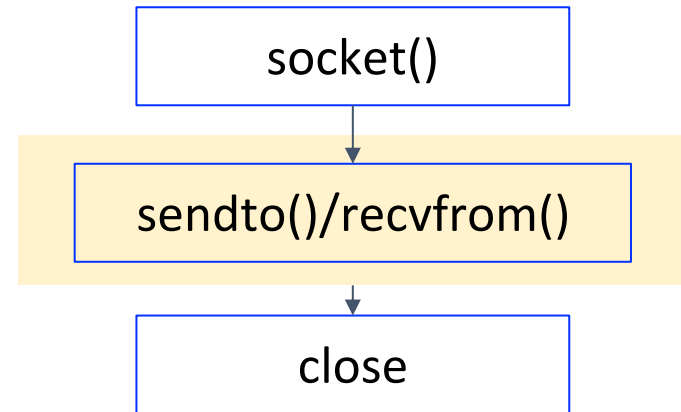


UDP socket programming (client)

TCP



UDP



UDP socket programming

src/server.cpp

```
#define MAXBUFSIZE 32 * 1024
```

creating socket

```
struct addrinfo *p; // populate this variable (AF_INET, SOCK_DGRAM)  
int sockfd = socket(p->ai_family, p->ai_socktype, p->ai_protocol);
```

increasing network buffer size

```
int bufferSize = MAXBUFSIZE;  
setsockopt(sockfd, SOL_SOCKET, SO_SNDBUF, &bufferSize, sizeof(bufferSize));  
setsockopt(sockfd, SOL_SOCKET, SO_RCVBUF, &bufferSize, sizeof(bufferSize));
```

binding socket to port

```
bind(sockfd, p->ai_addr, p->ai_addrlen);
```



UDP socket programming

src/server.cpp (continue)

```
struct sockaddr_storage their_addr;  
socklen_t addr_len = sizeof(their_addr);
```

recv data

```
char buf[MAXBUFSIZE];  
int numbytes = recvfrom(sockfd, buf, MAXBUFSIZE - 1, 0, (struct sockaddr*)&their_addr,  
&addr_len);  
buf[numbytes] = '\0';  
std::cout << buf << "\n";
```

send data

```
strcpy(buf, "hello, world!");  
numbytes = sendto(sockfd, buf, strlen(buf), 0, (struct sockaddr*)&their_addr,  
sizeof(their_addr));  
  
close(sockfd);
```



UDP socket programming

src/client.cpp

```
#define MAXBUFSIZE 32 * 1024
```

creating socket

```
struct addrinfo *p;
```

```
int sockfd = socket(p->ai_family, p->ai_socktype, p->ai_protocol);
```

increasing network buffer size

```
int bufferSize = MAXBUFSIZE;
```

```
setsockopt(sockfd, SOL_SOCKET, SO_SNDBUF, &bufferSize,  
sizeof(bufferSize));
```

```
setsockopt(sockfd, SOL_SOCKET, SO_RCVBUF, &bufferSize,  
sizeof(bufferSize));
```



UDP socket programming

src/client.cpp (continue)

send message

```
char* message = "start";
```

```
int numbytes = sendto(sockfd, message, sizeof(message), 0, p->ai_addr, p->ai_addrlen);
```

recv message

```
char buf[MAXBUFSIZE];
```

```
numbytes = recvfrom(sockfd, buf, MAXBUFSIZE - 1, 0, NULL, NULL);
```

```
buf[numbytes] = '\0';
```

```
std::cout << buf << "\n";
```

```
close(sockfd);
```



C++ and CMake

- **We will use C++ in this assignment**
- It makes life easier and OpenCV has official support for on C++
- CMake is a build system generator
- Prerequisites can be installed using `sudo apt install build-essential cmake`



CMake

src/main.cpp

```
#include <iostream>

int main(int argc, char* argv[])
{
    std::cout << "Hello, World\n";

    return 0;
}
```

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.5)
project(hello_world)

add_executable(main src/main.cpp)
```



CMake

- To generate build system: `cmake -B build`
- To build the project: `cmake --build build`
- Run the project: `build/main`



Video Streaming

- Video is moving images
- So streaming video means sending image frames in the video
- The server needs to read and send frames of the video one at a time and the client will read and display the frames **<more explanation>**



OpenCV

- Open Computer Vision library
- Utility functions which can be used to read video and extract frames
- OpenCV can be installed using the command `sudo apt install libopencv-dev libgtk-dev`
- Another library is need for video playback, install using `sudo apt install ffmpeg`



OpenCV

src/main.cpp

```
#include <iostream>
```

```
const std::string video_filename = "sintel-1280-surround.mp4";
```

```
int main(int argc, char* argv[])
```

```
{  
    cv::VideoCapture capture(video_filename);
```

```
if (!capture.isOpened())
```

```
{  
    std::cerr << "Error: Could not open video file: " << video_filename << "\n";  
    return 1;  
}
```



OpenCV

src/main.cpp (continue)

```
cv::Mat frame;
int key;
while(capture.isOpened())
{
    bool isSuccess = capture.read(frame);
    if (isSuccess)
    {
        std::cout << frame.type() << "\n";
        cv::imshow(video_filename, frame);
        if (key = cv::waitKey(30), key == 'q')
        {
            break;
        }
    }
}
```



OpenCV

src/main.cpp (continue)

```
else
{
    std::cerr << "Error: Could not get frame from video\n";
}
}

cv::destroyAllWindows();

return 0;
}
```



OpenCV

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.5)  
project(hello_world)
```

```
find_package(OpenCV REQUIRED)
```

```
include_directories(${opencv_include_dirs})  
add_executable(main src/main.cpp)  
target_link_libraries(main ${OpenCV_LIBS})
```



Encoding frames to reduce size

- The frame extracted from video is huge in size.
- One frame size: 2MB (1024x436)
- If sent as it is, it will cause congestion in the network.
- After encoding: ~50KB (40 times smaller)
- Packet will get dropped.
- Solution: encode/compress frame before sending.

```
std::vector<uchar> encodedFrame;  
cv::imencode(".jpg", frame, encodedFrame, {cv::IMWRITE_JPEG_QUALITY,  
encodeJPEGQuality});
```



Encoding frames to reduce size

- If the server encode frame before sending, then client has to receive and decode it.

```
cv::Mat decodedFrame;  
cv::imdecode(encodedFrame, cv::IMREAD_COLOR, &decodedFrame);
```



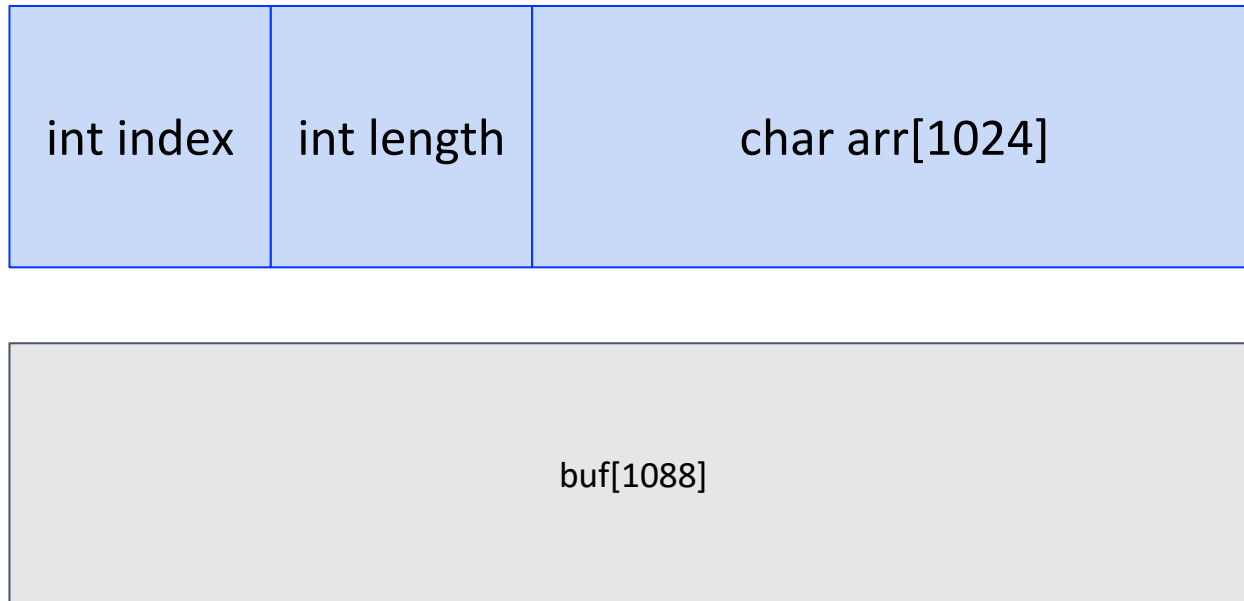
Sending and parsing binary data

- Till now we were sending data through sockets in plain text. They are bulky.
- Video frames and metadata should be sent in a binary format.
- You have to read the buffer and convert it into appropriate data structure.



Sending and parsing binary data

- If I want to send the following data, how will you send it?



Sending and parsing binary data

- We can use a data structure called *union*.

```
struct Data
{
    int index;
    int length;
    char arr[1024];
};

union U_Data
{
    struct Data data;
    char buf[sizeof(struct Data)];
};
```



Sending and parsing binary data

Encoding

```
union U_Data d;  
d.data.index = 0;  
d.data.length = 6;  
strcpy(d.data.arr, "hello");  
# now we can send d.buf through socket
```

Decoding

```
# let's say binary data is stored in buf, with length len  
union U_Data d2;  
memcpy( d2.buf, buf, len);  
std::cout << d2.data.index << "\n";
```



Thank you

